

METAROCKETC: Adaptive Encrypted Traffic Classification in Complex Network Environments via Time Series Analysis and Meta-Learning

Jianjin Zhao[✉], Graduate Student Member, IEEE, Qi Li[✉], Yueping Hong[✉], and Meng Shen[✉], Member, IEEE

Abstract—Encrypted Traffic Classification (ETC) is crucial for network security management and Quality of Service (QoS) improvement. There have been many attempts to tackle various ETC tasks, however, which generally suffer from task dependency and limited adaptability, falling short of meeting practical requirements. Under the realistic assumptions of complex network environments, diverse encryption techniques and ever-changing application landscapes coexist. It is highly desirable to learn the generic encrypted traffic representations to investigate the common knowledge across different ETC tasks and rapidly adapt to the dynamic shifts. To fill the gap, we propose METAROCKETC, a generic encrypted traffic classification framework, which extracts protocol-agnostic features to learn the common knowledge and rapidly adapt to novel ETC tasks and evolving network environments. In METAROCKETC, we first model packet length sequences of encrypted sessions as multivariate time series and perform random convolution kernel transformations to summarize discriminatory behavioral patterns across channels. By integrating METAROCKETC into an advanced Model-Agnostic Meta-Learning (MAML) framework, we learn a task-adaptive loss function to facilitate better generalization and transferability across diverse ETC tasks. Extensive experimental results demonstrate the superiority of METAROCKETC in both across-task and few-shot scenarios, highlighting its potential to provide a practical solution for encrypted traffic classification in real-world scenarios.

Index Terms—Encrypted traffic classification, time series analysis, meta-learning.

Manuscript received 29 May 2023; revised 28 October 2023; accepted 23 December 2023. Date of publication 5 January 2024; date of current version 15 April 2024. This work was supported in part by the National Natural Science Foundation of China under Grants 62172055, 61972039 and U20B2045; in part by the China National Funds for Excellent Young Scientists under Grant 62222201; in part by the Beijing Nova Program under Grants Z201100006820006 and 20220484174; in part by the Beijing Natural Science Foundation under Grants M23020, L222098 and 7232041; in part by the National Defense Basic Scientific Research Program of China under Grant JCKY2021602B002; in part by the Ningxia Natural Science Foundation under Grant 2022AAC03620; and in part by the BUPT Scientific and Technological Innovation Capacity Improvement Project under Grant 2021XD-A09. The associate editor coordinating the review of this article and approving it for publication was Y. Diao. (Corresponding author: Qi Li.)

Jianjin Zhao, Qi Li, and Yueping Hong are with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: jianjinzhao@bupt.edu.cn; liqi2001@bupt.edu.cn; hongyp@bupt.edu.cn).

Meng Shen is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: shenmeng@bit.edu.cn).

Digital Object Identifier 10.1109/TNSM.2024.3350080

I. INTRODUCTION

AS ONE of the most defining features of modern networks, the extensive deployment of encryption mechanisms in network communications plays a crucial role in preserving user privacy and ensuring data integrity. According to the Google Transparency Report [1], over 98% webpages were loaded over HTTPS in Chrome by platform in 2022. Yet, the proliferation of encrypted traffic brings great challenges to network traffic analysis, unveiling severe security concerns for potential hidden threats. Encrypted Traffic Classification (ETC), holding great significance for network security management and Quality of Service (QoS) improvement, and serving as a preliminary step in threat identification, has emerged as a critical area of interest for both industry and academia.

Meanwhile, there have been many attempts to address diverse ETC tasks (e.g., application classification, threat detection, device identification and website fingerprinting), achieving considerable progresses. For instance, Pham et al. [2] derive aggregated, temporal, statistical and categorical features to model the communication behaviors for mobile application classification. Fu et al. [3] aim to detect malicious traffic encrypted by TLS protocol and extract key information from TLS handshake messages such as TLS version and cipher suites. Sivanathan et al. [4] extract extensive statistical features and protocol-related features (e.g., DNS, NTP and TLS) to distinguish IoT devices. Wang et al. [5] leverage adversarial domain adaptation to fingerprint websites over few traffic. An emerging trend of ETC solutions appears to be more generic to handle different ETC tasks. Meng et al. [6] make an initial trial to learn general packet representation for diverse ETC tasks utilizing both encrypted and unencrypted data.

Unfortunately, prevalent research falls short in addressing the complexities inherent to practical network environments. Under realistic network assumption for encrypted traffic classification, diverse encryption techniques cater to different applications, raising the question: How to devise generic traffic representation for multiple ETC tasks? Moreover, with the continuous evolution of the network landscape, security solutions deployed ought to be adaptable to the corresponding dynamic shifts. In this context, we define complex network environments as those characterized by the coexistence of diverse encryption techniques and ever-changing application

landscapes. Previous studies generally share two common deficiencies in the face of complex network environments: (1) current features are often either overly excessive [4], [7] or protocol-dependent [8], [9], limiting their effectiveness against various encryption techniques (e.g., VPN, Tor and SSL/TLS); and (2) the model abilities are restricted to specific datasets or tasks [10], requiring frequent time-consuming retraining to tackle environment and task shift in practical applications.

To overcome the limitations and provide a practical solution for encrypted traffic classification, in this paper, we aim to extract protocol-agnostic features from encrypted sessions with minimal computational cost and develop a generic ETC framework with the ability to rapidly adapt to unknown tasks and evolving environments. We are motivated by recent advances in time series analysis [11], [12], [13] and meta-learning [14], [15], [16]. The per-packet feature sequences of encrypted sessions are naturally time series data. Also, meta-learning technique can enable models to rapidly adapt to novel tasks. To this end, we model per-packet feature sequences as multivariate time series and perform time series analysis to efficiently summarize sequential information for subsequent ETC tasks. Then we follow Model-Agnostic Meta-Learning (MAML), a meta-learning framework, to learn a task-adaptive loss function, achieving better generalization across diverse ETC tasks.

Formally, we propose a generic encrypted traffic classification framework, namely METAROCKETC, aimed at learning generic knowledge from different ETC tasks to build internal representation for encrypted sessions that are broadly suitable to tackle complex network environments. Specifically, we first extract packet length sequences from encrypted sessions and extend them as multivariate time series to enrich the representations. Then we perform random convolution kernel transformations on the multivariate time series by random channel sampling and summation operations to capture discriminatory behavioral patterns. To achieve better generalization, we build two meta-networks to learn a task-adaptive loss function during inner-loop optimization in the MAML manner. Extensive experimental results demonstrate the superiority of METAROCKETC in both across-task and few-shot scenarios.

In sum, the main contributions of this paper are summarized as follows:

- We enrich simple per-packet features (i.e., packet lengths and directions) as multivariate time series. Through random convolution kernel transformations, we effectively summarize the discriminatory behavioral patterns of encrypted traffic across different channels.
- We present a novel meta-learning framework to derive generic encrypted traffic representations. By designing a task-adaptive loss function, we investigate common knowledge across diverse ETC tasks and facilitate fast adaptation in complex network environments to satisfy the practical requirements.
- We develop a generic encrypted traffic classification framework, namely METAROCKETC, and comprehensively evaluate it on various datasets and scenarios.

Extensive experimental results demonstrate the superiority of METAROCKETC in both across-task and few-shot scenarios.

The remainder of this paper is organized as follows. In Section II, the related work is reviewed. In Section III, the preliminaries of our work is introduced. In Section IV, our proposed METAROCKETC is presented. The performance evaluations of METAROCKETC are provided in Section V. The limitations and potential future directions are discussed in Section VI. Finally, we conclude in Section VII.

II. RELATED WORK

In this section, we review the relevant studies in the following three aspects: encrypted traffic classification, time series classification and meta-learning.

A. Encrypted Traffic Classification

Encrypted traffic classification, aiming to classify the category of encrypted traffic from various applications, services or devices, is a crucial technique in network security management and QoS improvement. We briefly review those that are closely related to our work, which can be roughly divided into five categories according to analysis goals.

1) *Application Classification*: Mobile or Web application classification is crucial in traffic engineering. In this context, Taylor et al. [17] present APPSCANNER to fingerprint mobile applications from encrypted traffic. This approach derives 54 statistical features (e.g., minimum, maximum, mean and variance) per network flow and employ a Random Forest classifier for application classification. Liu et al. [18] propose FS-NET, an end-to-end ETC model following a multi-layer encoder-decoder structure. FS-NET learns sequential features from raw flow sequences (e.g., packet length and message type). Shen et al. [19] develop a Graph Neural Network (GNN)-based ETC model, namely GRAPHDAPP. This model represents packet length sequences as Traffic Interaction Graphs (TIGs), and employs graph classification to identify decentralized applications. Xu et al. [10] propose ETC-PS, which models packet length sequences of encrypted sessions as traffic paths and extracts path signature features for classification. The efficacy of ETC-PS has been demonstrated in various ETC tasks (e.g., HTTPS, VPN and Tor traffic classification).

2) *Threat Detection*: Early-stage studies on threat detection primarily focus on SSL/TLS traffic. Anderson et al. [8], [20], [21] first propose extracting features from unencrypted handshake messages for encrypted malicious traffic detection. This pioneering work has since served as a foundation for subsequent investigations. Zhao et al. [9] construct encrypted traffic graph to model the behavior similarities and service correlations among encrypted sessions, and employ an improved Graph Attention Network (GAT) to characterize the reliability and stability of encrypted sessions for threat detection in SSL/TLS traffic. Fu et al. [22], [23] present WHISPER, a real-time malicious traffic detection system by frequency domain analysis. Notably, the features employed in WHISPER are not

TABLE I
COMPARISON BETWEEN OUR WORK AND THE EXISTING STUDIES

Analysis Goal	Ref.	Feature Extraction	Various ETC tasks	Protocol Agnostic	w/o Re-training	Task Adaptive
Application Classification	[17]–[19] [10]	packet lengths	✗ ✓	✗ ✓	✗ ✗	✗ ✗
Threat Detection	[8], [9], [20], [21] [23]	TLS handshake messages packet lengths & message types	✗ ✗	✗ ✓	✗ ✗	✗ ✗
Device Identification	[4], [7] [24], [25]	TLS & DNS & NTP packet lengths	✗ ✗	✗ ✓	✗ ✗	✗ ✗
Website Fingerprinting	[26]–[28]	packet lengths	✗	✓	✗	✗
Generic	[6] Our Work	raw bytes packet lengths	✓ ✓	✓ ✓	✓ ✓	✗ ✓

confined to specific protocols, rendering it more versatile. WHISPER has achieved satisfactory detection accuracy and efficiency across a range of datasets.

3) *Device Identification*: Network devices, typically IoT devices, are mass deployed in contemporary era. It is necessary for network administrators to be aware of connected IoT devices and monitor corresponding network behaviors. Sivanathan et al. [4] extract extensive features, including volume and time distribution, DNS and NTP queries and cipher suites in SSL/TLS handshake messages to distinguish the network behaviors of various IoT devices. Duan et al. [24] form the frequency distribution of unique packet lengths as features and utilize k -NN algorithm for IoT device identification. Kostas et al. [7] propose IOTDEVID, which integrates feature selection techniques and a genetic algorithm to rigorously select features, achieving high identification accuracy of IoT devices at packet level. Zhao et al. [25] consider simple per-packet features and design a shuffle&split organization scheme to tackle the data imbalance problem in IoT environment and perform time series analysis to realize efficient IoT device identification.

4) *Website Fingerprinting*: Website fingerprinting poses a significant threat by potentially exposing the sensitive information regarding users' browsing behaviors within encrypted traffic. Rimmer et al. [26] conduct a comprehensive exploration of deep learning techniques to learning traffic patterns for website fingerprinting. Sirinam et al. [27] leverage a robust architecture of Convolutional Neural Network (CNN) to perform website fingerprinting attacks against defenses. Shen et al. [28] extract distinctive block, sequence and statistical features for efficient fine-grained website fingerprinting. Sirinam et al. [29] propose to utilize triplet networks for N -shot learning in website fingerprinting to tackle the adverse effects of realistic scenarios. Wang et al. [5] further perform website fingerprinting over few encrypted traffic by leveraging adversarial domain adaptation to be more practical.

5) *Generic*: There have been several efforts not limited to specific ETC tasks but aiming to learn common knowledge with great generalization potential. Meng et al. [6] introduce a novel contrastive learning paradigm to derive generic packet representation for various traffic classification tasks. Guarino et al. [30] delve into meta-learning, transfer learning and contrastive learning to foster better representation learning

in encrypted traffic analysis. Shen et al. [31] present a comprehensive survey on machine learning-based encrypted traffic analysis, highlighting the prevailing challenge of achieving generalization and transferability. Due to network heterogeneity, parameter variation and temporal drift, it is highly desirable that the classifier can be flexibly transferred to different datasets and tasks.

Unfortunately, a common limitation of many existing studies is that the extracted features are often too complex and protocol-dependent, rendering them unable to tackle different encryption techniques. While some studies utilize generic features and evaluate model performance on different ETC tasks, they typically retrain the model on different datasets, which is really resource-consuming and impractical in real-world environments.

To intuitively present the superiority of our work, we map the key differences in Table I. Specifically, we take advantages of time series analysis and meta-learning to develop METAROCKETC, a generic encrypted traffic classification framework that achieves exceptional generalization across various ETC tasks. Rather than frequently retraining, METAROCKETC learns common knowledge from different ETC tasks and can rapidly adapt to unknown ETC tasks and evolving environments, providing a more practical solution for encrypted traffic classification.

B. Time Series Classification

Time series data, which consists of quantitative values arranged in chronological order, is ubiquitous across various domains. Numerous studies have focused on Time Series Classification (TSC) problems by analyzing ordered attributes. As discussed in [32], these approaches can be roughly divided into five categories: distanced-based, interval-based, shapelet-based, dictionary-based and ensemble approaches. Distance-based approaches typically combine distance metrics with a One Nearest Neighbor (1-NN) classifier as the baseline, where many novel distance metrics including Dynamic Time Warping (DTW) distance and its variants [33], [34], [35] are proposed. Interval-based approaches [36], [37] identify those intervals with discriminatory sub series instead of using the whole series. Shapelets [38] denote the location-irrelevant, representative sub series for each class. Relevant

studies [39], [40] primarily focus on efficient shapelet discovery. Dictionary-based approaches count the frequencies of repetitive sub series to distinguish time series. Ensemble approaches [41] integrate multiple types of the above models to achieve better performance. In addition to these studies, ROCKET [11] and its variants MINIROCKET [12] and MULTIROCKET [13] have emerged as revolutionary TSC models for their high overall accuracy and remarkably fast training time. In this work, we utilize the multivariate variant of MULTIROCKET to capture the discriminatory behavioral patterns of encrypted sessions.

C. Meta-Learning

Meta-learning, or learning to learn, has emerged as a prominent solution for learning common prior knowledge across tasks and facilitating rapid adaptation to new tasks, which has achieved considerable success in the realm of few-shot learning applications. One such technique, recognized for its simplicity and state-of-the-art performance, is Model-Agnostic Meta-Learning (MAML) [14]. MAML aims to learn good initialization parameters for the base model such that after applying a small number of gradient steps with respect to the support set, the adapted model can achieve strong generalization performance on the query set. MAML is a simple yet elegant optimization-based meta-learning framework that can be integrated with any gradient-based models. Subsequent works have built on the foundation of vanilla MAML, proposing a series of improvements to achieve better generalization. Antoniou et al. [15] propose various modifications on MAML to improve the stability, generalization performance, convergence speed and computational overhead. Baik et al. [42], [43] further enhance the fast adaptation of MAML by learning adaptive hyper-parameters or loss function. In this work, we resort to an advanced MAML framework by learning task-adaptive loss functions for the backbone neural networks to acquire generic knowledge across various ETC tasks. This work is designed to be generic and capable of rapidly adapting to unknown ETC tasks and dynamic environments.

III. PRELIMINARIES

In this section, we present the task definition in general meta-learning algorithms, briefly introduce MAML, one of the most popular optimization-based meta-learning approach, and provide the problem formulation of our work in detail. The notations used in this paper are summarized in Table II.

A. Task Definition

Meta-learning algorithms typically assume a distribution of tasks, denoted as $p(\mathcal{T})$, from which a collection of tasks $\{\mathcal{T}_i\}_{i=1}^T$ is sampled. Each task $\mathcal{T}_i$ consists of a support set $\mathcal{D}_i^S$ for the base model to obtain task-specific knowledge and a query set $\mathcal{D}_i^Q$ for the adapted model to evaluate the generalization performance. Both the support set and the query set comprise pairs of inputs and outputs: $\mathcal{D}_i^S = \{(x_i^s, y_i^s)\}_{s=1}^K$ and $\mathcal{D}_i^Q = \{(x_i^q, y_i^q)\}_{q=1}^M$.

TABLE II
NOTATIONS IN THIS PAPER

Notation	Description
$\mathcal{T}_i$	The i_{th} task for model adaptation and evaluation
$\mathcal{D}_i^S, \mathcal{D}_i^Q$	The support set and query set of task $\mathcal{T}_i$
J	The total number of inner-loop updates
α, β	The inner-loop and outer-loop learning rates
$\theta/\theta_{i,0}$	The initial parameters for the base model
$\theta_{i,j}$	The parameters for the base model at step j on task $\mathcal{T}_i$
$\theta_i/\theta_{i,j}$	The task-specific parameters for the base model on task $\mathcal{T}_i$
ϕ	The parameters of learnable loss function
ψ	The parameters of loss function adapter
f_θ	The base model parameterized by θ
$\mathcal{L}_\phi$	The learnable loss function parameterized by ϕ
$\mathcal{G}_\psi$	The loss function adapter parameterized by ψ
γ, η	The parameters for affine transformations
$t_{i,j}$	The task state for task $\mathcal{T}_i$ at step j
X, U, D, U', D'	The original and transformed packet length sequences
Y	The transformed multivariate time series
Z	The convolution output after transformations

For a more rigorous evaluation of meta-learning models, all tasks are split into two distinct sets: the meta-training set and the meta-testing set. It should be noted that both the two sets do not share any tasks. Meta-learning models acquire the ability of adapting to new tasks during the meta-training phase and evaluate their generalization performance during the meta-testing phase.

B. Model-Agnostic Meta-Learning

As perhaps the most popular optimization-based meta-learning algorithm, MAML has achieved remarkable success across diverse learning problems. Its objective is to learn good initialization parameters for the base model, enabling the adapted model to perform well on new tasks after only a few gradient updates on a few-shot dataset.

Formally, MAML takes a network f_θ with parameters θ as the base model. It adopts bi-level optimization framework including inner-loop and outer-loop optimizations to learn the best initialization for the base model.

Inner-loop Optimization: Given task $\mathcal{T}_i$ and its associate support set $\mathcal{D}_i^S$, the base model is fine-tuned using its initial weights θ . This task-specific tuning involves a fixed number of updates via gradient descent. Specifically, each task undergoes J inner-loop updates in total. At step j , the inner-loop optimization is denoted as:

$$\theta_{i,j+1} = \theta_{i,j} - \alpha \nabla_{\theta_{i,j}} \mathcal{L}(\mathcal{D}_i^S; \theta_{i,j}), \quad (1)$$

where $\theta_{i,0}/\theta$ denotes the initial model weights, α denotes the inner-loop learning rate and $\mathcal{L}(\mathcal{D}_i^S; \theta_{i,j})$ is the loss on the support set $\mathcal{D}_i^S$ after j updates. Once J inner-loop optimization steps are completed, we obtain task-specific weights $\theta_i/\theta_{i,j}$ for each task $\mathcal{T}_i$.

Outer-loop Optimization: Given task $\mathcal{T}_i$ and its associate query set $\mathcal{D}_i^Q$, the initialization parameters θ are evaluated using the task-specific parameters θ_i on the unseen query set. The outer-loop update serves as the feedback for updating the initialization weights θ , facilitating the model to learn a better initialization that can generalize across various tasks, which is

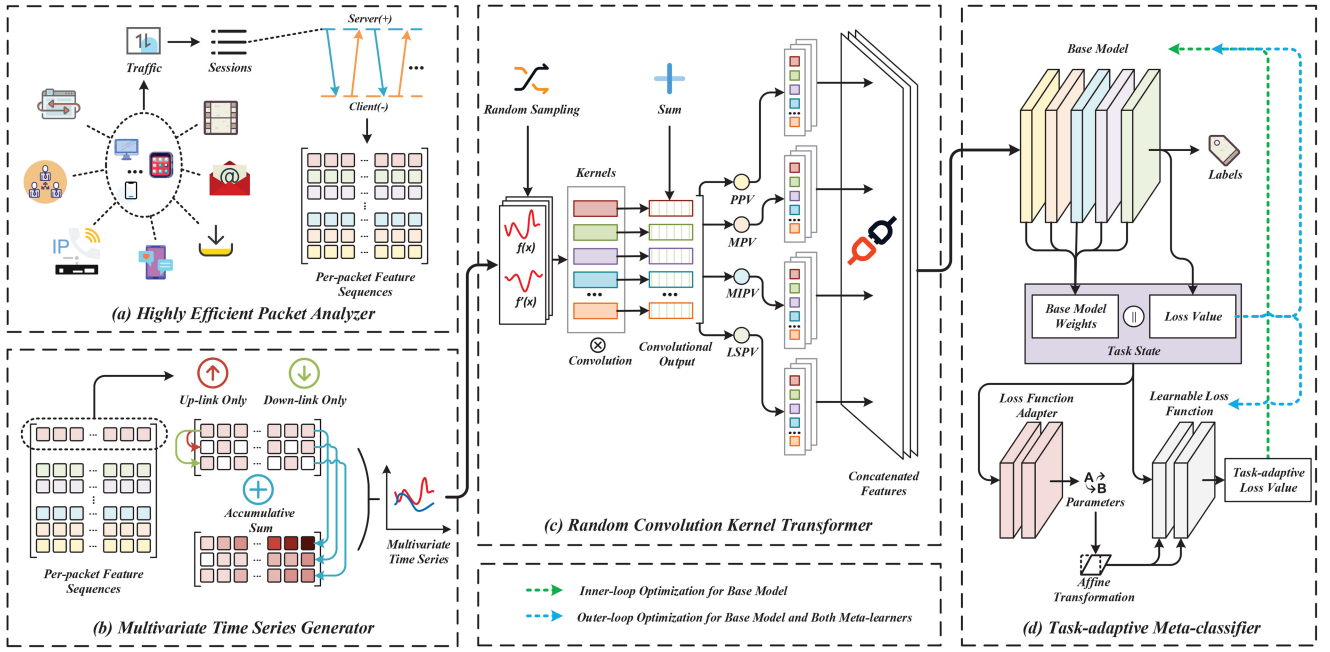


Fig. 1. The overall framework of METAROCKETC: (a) Highly Efficient Packet Analyzer; (b) Multivariate Time Series Generator; (c) Random Convolution Kernel Transformer; (d) Task-adaptive Meta-classifier.

denoted as:

$$\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\mathcal{T}_i} \mathcal{L}(\mathcal{D}_i^Q; \theta_i), \quad (2)$$

where β denotes the outer-loop learning rate and $\mathcal{L}(\mathcal{D}_i^Q; \theta_i)$ is the loss on the query set $\mathcal{D}_i^Q$ with task-specific weight θ_i .

C. Problem Formulation

In this paper, we aim to investigate common encrypted traffic representation that are broadly suitable to various ETC tasks and can rapidly adapt to the dynamic shift in complex network environments. Under realistic network assumptions, ETC models may severely degrade due to network heterogeneity, parameter variations and temporal drift, necessitating frequent and resource-intensive model retraining to meet practical needs.

Meta-learning has recently emerged as a promising approach to enable model to learn common prior knowledge shared across tasks and rapidly generalize to novel tasks under few-shot regime. We consider utilizing meta-learning to develop a generic and practical ETC solution that not only generalizes well to unknown ETC tasks, but also rapidly adapts to the evolving network environments.

Our proposed solution, namely METAROCKETC should exhibit proficiency in both across-task and few-shot scenarios. In across-task evaluation, METAROCKETC extracts common knowledge from ETC task A using meta-learning, and then transfers the extracted knowledge to ETC task B . The transferability to unknown ETC tasks can be evaluated by performance comparison under different label rates. In few-shot evaluation, METAROCKETC is directly assessed on specific ETC tasks in

N -way k -shot settings to evaluate the performance in few-shot regime.

To achieve this, we consider protocol-agnostic per-packet features of encrypted sessions as natural time series data. We utilize random convolution transformations to extract multi-scale communication patterns as raw features, which requires no training process and is highly efficient. Then we follow an improved MAML framework to learn task-adaptive loss function to learn genetic representations investigating common knowledge and enabling rapid adaptation to unknown ETC tasks.

IV. METHODOLOGY

A. Framework Overview

In this section, we present our generic encrypted traffic classification framework, METAROCKETC. METAROCKETC first extracts per-packet feature sequences and extends these sequences to generate multivariate time series. Then it performs a series of random convolution kernel transformations to capture the local and global sequential information of the multivariate time series from the ordered packets. To tackle the complex and time-evolving network environments, we design a task-adaptive meta-classifier to achieve better generalization and transferability across various ETC tasks. The performance of METAROCKETC is evaluated in across-task and few-shot scenarios. As illustrated in Fig. 1, METAROCKETC can be mainly divided into the following four modules.

Highly Efficient Packet Analyzer processes high-speed network traffic in complex network environments, aggregates packets into sessions and extracts per-packet feature sequences. As a generic encrypted traffic classification framework, METAROCKETC is not restricted to specific encryption

protocols. To improve the efficiency and versatility of METAROCKETC, the packet analyzer does not extract any features related to the application layer. In this work, we only extract the bidirectional packet lengths as per-packet features.

Multivariate Time Series Generator extends per-packet feature sequences by several transformations including uplink/downlink-only packet reservation and cumulative sum calculation, to obtain supplementary information and reveal the underlying dynamics of network behaviors. These extended feature sequences are modeled as multivariate time series for subsequent transformations. The details of the multivariate time series generator are presented in Section IV-B.

Random Convolution Kernel Transformer first generates a collection of kernels with parameters initialized in a deterministic way, and then performs convolution operations. For multivariate time series, the channels are randomly selected per kernel during convolution operation and the results are summed up to form the final convolution output. To capture the discriminatory behavioral patterns of encrypted sessions, each convolution output is summarized with four pooling operators. We will comprehensively elaborate on the random convolution kernel transformations in Section IV-C.

Task-adaptive Meta-classifier integrates a learnable loss function into the inner-loop optimization in MAML. It builds two meta-learners to perceive the state of the specific task, where one meta-learner for learning a loss function and the other for generating affine transformation parameters to transform the learnable loss function to be task-adaptive. During fine-tuning phase, a transfer learning technique namely freezing is utilized to facilitate rapid adaptation. Following the improved MAML framework, METAROCKETC learns common knowledge from different ETC tasks to build the internal representation that are broadly suitable to tackle complex network environments. The details of the task-adaptive meta-classifier are described in Section IV-D.

B. Multivariate Time Series Generator

The per-packet feature sequences of encrypted sessions are naturally time-dependent variables, therefore, we acquire them from the highly efficient packet analyzer and model them as time series data. Moreover, existing studies [10] have demonstrated that the raw feature sequences are insufficient to achieve accurate encrypted traffic classification and proposed some tricks (e.g., cumulative sum calculation and unidirectional packet reservation) to enrich feature sequences. In this work, we also perform some transformations to enrich the feature sequences and models them as multivariate time series.

Specifically, we reserve the first N packets for each encrypted session and extract packet length sequence as univariate time series. Then we reserve uplink and downlink packets respectively and calculate their cumulative sums of packet length sequences as multivariate time series to enrich the expressive power. In each encrypted session, the lengths of uplink and downlink packets are respectively set as positive (+) and negative (−) to indicate the directions. Given the original packet length sequence $X = \{x_1, x_2, \dots, x_N\}$, the

uplink/downlink-only packet length sequences (U/D) are denoted as:

$$U = \{u_1, u_2, \dots, u_N\}, u_i = \begin{cases} x_i & x_i > 0 \\ 0 & x_i \leq 0 \end{cases} \quad (3)$$

and

$$D = \{d_1, d_2, \dots, d_N\}, d_i = \begin{cases} 0 & x_i > 0 \\ x_i & x_i \leq 0, \end{cases} \quad (4)$$

where the packet lengths in opposite directions are set to zero.

With the original X , uplink-only U and downlink-only D , we then calculate their corresponding cumulative sums X' , U' and D' respectively by:

$$X' = \text{CS}(X) = \{x'_1, x'_2, \dots, x'_N\}, x'_i = \sum_{j=1}^i x_j, \quad (5)$$

$$U' = \text{CS}(U) = \{u'_1, u'_2, \dots, u'_N\}, u'_i = \sum_{j=1}^i u_j, \quad (6)$$

$$D' = \text{CS}(D) = \{d'_1, d'_2, \dots, d'_N\}, d'_i = \sum_{j=1}^i d_j, \quad (7)$$

where the function $\text{CS}(\cdot)$ is to calculate the cumulative sum of given packet length sequences, shedding light on the specific payloads of content delivery or communication patterns associated with particular websites or services.

In this way, we transform the original univariate time series X into multivariate time series $Y = [X; U; D; X'; U'; D']$ for subsequent analysis. Here we take *ISCX-VPN* dataset as an example and randomly sample an encrypted session to illustrate the interactions and corresponding transformations in Fig. 2.

C. Random Convolution Kernel Transformer

In this module, we extract discriminatory behavioral patterns from the multivariate time series by random convolution transformations. Specifically, we first generate a large number of random convolution kernels, which are effective in capturing both local and global information at multiple scales. Then we perform the transformations on the multivariate time series in parallel. For multivariate time series, the channels are randomly selected per kernel during calculation and the corresponding convolution output are summed up for aggregation. Finally, we derive the features with multiple pooling operators to summarize the convolution outputs.

1) *Convolution Kernels*: It has been demonstrated that though random convolution kernels are highly effective, comparable performance can be achieved in a more deterministic and efficient way. To this end, our METAROCKETC follows the most-deterministic setups in [12], [13]. The details of the convolution kernels are as follows:

- *Length and Weights*: Kernel length is fixed to 9, with weights restricted to two values (−1 and 2), where three weights have the value 2 and six weights have the value −1. There are in total $C_9^3 = 84$ fixed kernels determined for subsequent transformations.

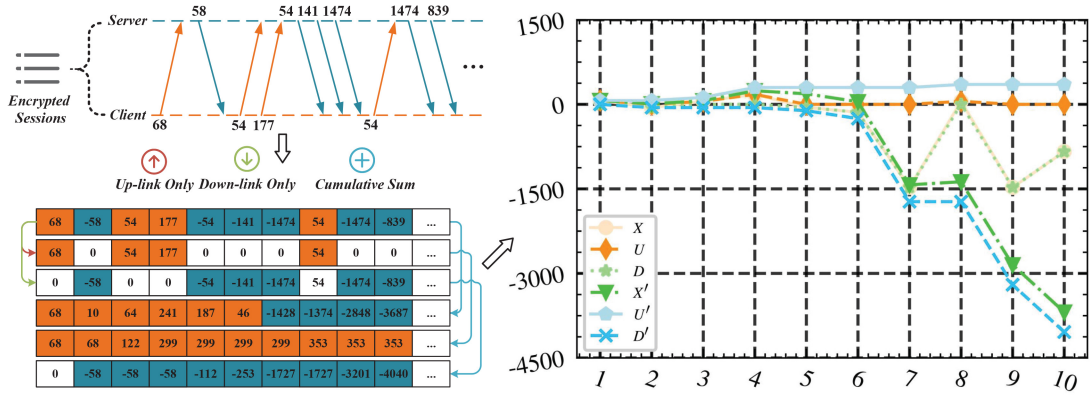


Fig. 2. An example of multivariate time series generation from ISCX-VPN dataset.

- **Dilation:** To expand the receptive field during convolutions, all kernels share a fixed set of dilations in the range $\{[2^0], \dots, [2^{\max}]\}$, with the exponent d drawn from a uniform distribution $\mathcal{U}(0, \max = \log_2 \frac{|x|-1}{|w|-1})$, where $|x|$ is input length, and $|w|$ is kernel length (i.e., 9).
- **Bias:** For each kernel&dilation combination, we randomly sample one multivariate time series example and randomly select the channels to compute the convolution outputs and sum them up as the final output, the quantiles of which is taken as corresponding bias values.
- **Padding:** Padding is alternated between kernel/dilation combinations, such that half combinations enable zero padding and half disable.
- **Stride:** Stride is fixed to one to ensure every data point of the time series is traversed.

2) **Convolution Transformations:** With the generated convolution kernels, we perform transformations on the multivariate time series to extract multi-scale patterns. Given multivariate time series Y , convolution kernel with weight W , dilation d and bias b , and selected channels C , the convolution operation is denoted as:

$$z_{c,i} = W * y_{c,i} = \left(\sum_{j=0}^{|w|-1} y_{c,(i+j \times d - \frac{|w|-1}{2} \times d)} \times w_j \right) + b, \quad (8)$$

where $z_{c,i}$ is the convolution output Z at position i of channel $c \in C$ computed by sliding dot production.

Then we sum up the convolution output of each channel to obtain the final convolution output of multivariate time series as:

$$Z = \{z_1, z_2, \dots\}, z_i = \sum_{c \in C} z_{c,i}. \quad (9)$$

3) **Pooling Operators:** Previous studies have demonstrated that diversity is the key to improve accuracy in time series classification. To obtain diverse representations, multiple pooling operators are utilized to summarize the convolution outputs from different perspectives. For each convolution output Z , four pooling operators are applied to compute features, including Proportion of Positive Values (PPV), Mean of Positive

Values (MPV), Mean of Indices of Positive Values (MIPV) and Longest Stretch of Positive Values (LSPV).

PPV is first introduced in ROCKET [11] and has been demonstrated to be exceptionally effective, which flexibly reflects the proportion of matches between the input and the given pattern. Whether the match is strong or weak depends on the bias value. PPV is computed using Eq. (10).

$$\text{PPV}(Z) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}(z_i > 0), \quad (10)$$

where n is the length of convolution output Z , and $\mathbb{I}(\cdot)$ is an indicator function such that $\mathbb{I} = 1$ if the condition $(z_i > 0)$ is true, otherwise $\mathbb{I} = 0$.

MPV, MIPV and LSPV are three further pooling operators proposed in MULTIROCKET [13]. MPV computes the mean of the positive values in the convolution output, similarly to PPV, which flexibly reflects the intensity of matches between the input and the given pattern depending on the bias value. MPV is computed using Eq. (11).

$$\text{MPV}(Z) = \frac{1}{m} \sum_{i=1}^m z_i^+, \quad (11)$$

where m is the number of positive values in convolution output Z . MIPV computes the mean of indices of positive values, which captures the relative locations of positive values in convolution output Z . MIPV is computed using Eq. (12).

$$\text{MIPV}(Z) = \begin{cases} \frac{1}{m} \sum_{i=1}^m I(z_i^+) & \text{if } m > 0 \\ -1 & \text{otherwise,} \end{cases} \quad (12)$$

where $I(\cdot)$ is a function that returns the index of z_i^+ in convolution output Z . LSPV counts the length of the longest sub-sequences of positive values in convolution output Z . LSPV is computed using Eq. (13).

$$\text{LSPV}(Z) = \max [j - i \mid \forall i \leq k \leq j, z_k > 0]. \quad (13)$$

The four pooling operators can effectively summarize the convolution outputs for subsequent analysis and can be computed with a slight overhead to meet the practical requirements. Here we provide a dummy example to present the results of the four pooling operators with different convolution outputs as shown in Table III.

TABLE III
A DUMMY EXAMPLE TO PRESENT POOLING OPERATORS WITH
DIFFERENT CONVOLUTION OUTPUTS

Convolution Output	PPV	MFV	MIPV	LSPV
[0,0,0,0,1,1,1,1]	0.5	1	5.5	4
[1,1,0,0,1,1,0,0]	0.5	1	2.5	2
[1,1,1,1,1,1,1,1]	1	1	3.5	8
[9,9,9,9,0,0,0,0]	0.5	9	1.5	4

D. Task-Adaptive Meta-Classfier

1) *Overview*: To handle the complex network environments, where large volumes of encrypted traffic contain diverse sessions of different categories (e.g., applications, services and encryption protocols), task-adaptive meta-classifier leverages the meta-learning technique for learning common knowledge across various ETC tasks as generic encrypted traffic representations.

Following bi-level optimization framework in MAML, it mainly comprise one base model and two meta-learners. The base model takes the discriminatory behavioral patterns obtained from the previous module as input and further compress these patterns into lean features for classifications. The two meta-learners replace the traditional fixed loss function and take tasks states as input to serve as the task-adaptive loss function in the inner-loop optimization. Both meta-learners are modeled using neural networks and take task states as input to enable task-adaptive loss function, where one meta-learner parameterized by ϕ serves as a learnable loss function $\mathcal{L}_\phi(\cdot)$, and the other meta-learner parameterized by ψ serves as a loss function adapter $\mathcal{G}_\psi(\cdot)$.

Specifically, $t_{i,j}$ denotes the task state for task $\mathcal{T}_i$ at inner-loop update step j , represented by the general loss values on support set $\mathcal{D}_i^S$ and the weights of the base model. The loss function adapter $\mathcal{G}_\psi(\cdot)$ first takes task state $t_{i,j}$ as input and generates parameters γ and η for updating the learnable loss function $\mathcal{L}_\phi(\cdot)$ to be task-adaptive as denoted in Eq. (14).

$$(\gamma, \eta) = \mathcal{G}_\psi(t_{i,j}), \quad (14)$$

where γ and η are affine transformation parameters used to transform the learnable loss function, as denoted in Eq. (15).

$$\phi' = \gamma\phi + \eta. \quad (15)$$

In this way, the inner-loop update of the original MAML in Eq. (1) is modified to incorporate task states and update the base model as denoted in Eq. (16).

$$\theta_{i,j+1} = \theta_{i,j} - \alpha \nabla_{\theta_{i,j}} \mathcal{L}_{\phi'}(t_{i,j}), \quad (16)$$

Correspondingly, the outer-loop update of the original MAML in Eq. (2) is also modified to update both the base model and meta-learners as denoted in Eq. (17).

$$(\theta, \phi, \psi) = (\theta, \phi, \psi) - \beta \nabla_{(\theta, \phi, \psi)} \sum_{\mathcal{T}_i} \mathcal{L}(\mathcal{D}_i^Q; \theta_i), \quad (17)$$

where the initialization parameters θ , ϕ and ψ of the base model and meta-learners are optimized to encode common prior knowledge. Moreover, task-adaptive strategy during

fine-tuning phase is designed to enable the task-adaptive meta-classifier to quickly adapt to different ETC tasks. The detailed workflow of the task-adaptive meta-classifier is illustrated in Fig. 3.

2) *Backbone Architecture*: In the previous module, we adopt MULTIROCKET [13] to capture the discriminatory behavioral patterns within encrypted sessions, which employs an ensemble of numerous random convolution kernels, offering remarkable efficiency for the elimination of training requirements. However, the utilization of an excessive number of random kernels may lead to high feature dimensionality and potential redundancy. To address this issue, we employ 4-layer MLP (denoted as 4-MLP hereafter) with Rectified Linear Unit (ReLU) activation as the base model backbone architecture for feature encoding. Additionally, we apply batch normalization following ReLU activation to mitigate internal co-variate shift and enhance generalization performance.

Let h^l denote the input to the l -th layer of 4-MLP, which is parameterized by weights $\mathbf{w}$ and bias $\mathbf{b}$. The normalization operation is denoted by $\mathbf{BN}(\cdot)$ and the ReLU activation is denoted by $\mathbf{ReLU}(\cdot)$. Consequently, the output h^{l+1} of the l -th layer can be denoted as:

$$h^{l+1} = \mathbf{BN}(\mathbf{ReLU}(\mathbf{w}h^l + \mathbf{b})). \quad (18)$$

3) *Meta-Learner Design*: To enable the meta-classifier to achieve better generalization across various ETC tasks, we substitute the traditional fixed loss function with a task-adaptive loss function via two meta-learners. It has been demonstrated that learning a better loss function for the inner-loop optimization is essential for achieving superior generalization in MAML. The meta-learners perceive task states to produce task-adaptive loss values on the support set during the inner-loop optimization. The task state $t_{i,j}$ is formulated as the concatenation of mean of support set losses $\mathcal{L}(\mathcal{D}_i^S; \theta_{i,j})$ and layer-wise mean of base model weights $\theta_{i,j}$.

Specifically, both meta-learners are modeled by 2-layer MLP with ReLU activation and take the task state as input. One serves as learnable loss function $\mathcal{L}_\phi$, returning a scalar value as loss value, while the other serves as loss function adapter $\mathcal{G}_\psi$, producing parameters to perform layer-wise affine transformations on the loss function parameters ϕ . The overall training procedure of the task-adaptive meta-classifier is outlined in Algorithm 1.

4) *Task-Adaptive Strategy*: During the meta-training stage, the task-adaptive meta-classifier acquires common knowledge from various ETC tasks, building the internal representations that are broadly applicable to complex network environments. Here we provide a task-adaptive strategy that employs freezing, a straightforward transfer learning technique to facilitate rapid adaptation to new or similar ETC tasks.

More specifically, we initially take the first few layers from the previously trained meta-classifier and freeze them to prevent compromising the weights and damaging the contained information during subsequent training iterations. Next, we add some new trainable layers atop the frozen layers and learn to transform the previous features into predictions for

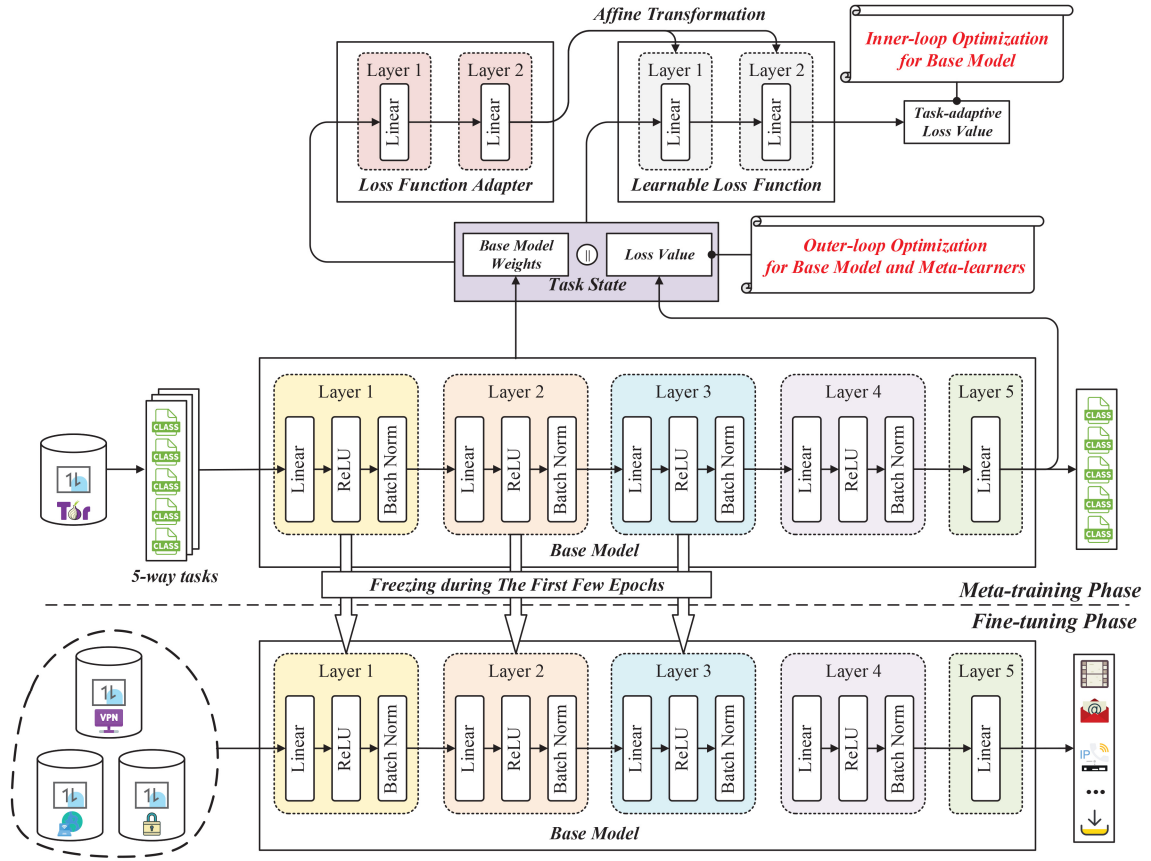


Fig. 3. The detailed workflow of the task-adaptive meta-classifier. During meta-training phase, following MAML framework, it acquires common prior knowledge from *ISCX-Tor* dataset in 5-way k -shot setting. In the inner-loop optimization, two meta learners respectively serve as the loss function adapter and learnable loss function to return task-adaptive loss value for updating the base model. In the outer-loop optimization, the two meta-learners and the base model are updated according to the original loss value. During fine-tuning phase, the well-trained base model are fine-tuned on specific datasets (i.e., *ISCX-VPN*, *CTU*, and *BetterNet-HTTPS* datasets) and the freezing technique is applied for rapid adaptations on new ETC tasks.

new ETC tasks. We propose freezing the initial layers for the first few epochs of training phase, and then unfreezing them to retrain the meta-classifier. In this way, the meta-classifier progressively adapts the previous features to new ETC tasks, achieving significant improvements in efficiency. Notably freezing is applicable in both across-task and few-shot scenarios. See Section V for more details.

V. EVALUATION

A. Experimental Setup

1) *Dataset*: To comprehensively evaluate the performance of METAROCKETC, we select six datasets: *ISCX-VPN*, *ISCX-Tor*, *CTU*, *5GAD*, *Google-Home*, and *BetterNet-HTTPS* datasets, all of which are publicly available and extensively applied for the evaluations of various ETC tasks. Table IV summarizes the details of these datasets.

ISCX-VPN [44] dataset is a Virtual Private Networks (VPNs) traffic dataset, including 7 categories (i.e., Browsing, Email, Chat, Streaming, FTP, VoIP and P2P) in both VPN and non-VPN.

ISCX-Tor [45] dataset is an Onion Router (Tor) traffic dataset, including 7 categories (i.e., Browsing, Email, Chat, Streaming, FTP, VoIP, P2P) in both Tor and non-Tor.

Datasets	#PCAP	#Session	#Class
<i>ISCX-VPN</i>	171	1,761	12
<i>ISCX-Tor</i>	126	5,340	12
<i>CTU</i>	22	2,655	59
<i>5GAD</i>	82	59,445	2
<i>Google-Home</i>	~	149,745	100
		42,750	950
<i>BetterNet-HTTPS</i>	24	(closed-world)	(closed-world)
		65,141	2,269
		(open-world)	(open-world)

CTU [46] dataset is a HTTPS traffic dataset, including the network traces during accessing Alexa Top 1000 websites.

5GAD [47] dataset consists of intercepted 5G network data by a third-party attacker, including both normal and malicious traffic. The data was generated in a simulated environment and collected via an Internet-connected Linux machine running a 5G Core (5GC) network implemented with the open source free5GC software.

Google-Home [48] dataset is a large-scale dataset collected on Google Home. Using 5 automated voices rendered by public text-to-speech APIs, *Google-Home* dataset consists of 150,000 traffic traces including 100 commands/classes and 1,500 traffic traces per class.

Algorithm 1: The Workflow of Task-Adaptive Meta-Classifier

Require: Task distribution $p(\mathcal{T})$
Require: Inner-loop and outer-loop learning rates α, β
Require: Base model f_θ , meta-learners $\mathcal{G}_\psi, \mathcal{L}_\phi$

```

1 Randomly initialize parameters  $\theta, \psi$  and  $\phi$ 
2 while not done do
3   Sample a batch of tasks  $\mathcal{T}_i \sim p(\mathcal{T})$ 
4   for all  $\mathcal{T}_i$  do
5     Retrieve support set  $\mathcal{D}_i^S = \{(x_i^s, y_i^s)\}_{s=1}^K$ , query
6     set  $\mathcal{D}_i^Q = \{(x_i^q, y_i^q)\}_{q=1}^M$  from  $\mathcal{T}_i$ 
7     Initialize  $\theta_{i,0} = \theta$ 
8     for  $j$  in number of inner-loop updates  $J$  do
9       Compute base model outputs on support set:
10       $f_{\theta_{i,j}}(\mathcal{D}_i^S) = \{f_{\theta_{i,j}}(x_i^s)\}_{s=1}^K$ 
11      Compute loss values on support set:
12       $\mathcal{L}(\mathcal{D}_i^S; \theta_{i,j}) = \mathcal{L}(f_{\theta_{i,j}}(x_i^{s=1:K}), y_i^{s=1:K})$ 
13      Perceive task state:  $t_{i,j} = [\mathcal{L}(\mathcal{D}_i^S; \theta_{i,j}) || \theta_{i,j}]$ 
14      Compute affine transformation parameters:
15       $\gamma_{i,j}, \eta_{i,j} = \mathcal{G}_\psi(t_{i,j})$ 
16      Transform loss function to be task-adaptive:
17       $\phi'_{i,j} = \gamma_{i,j}\phi + \eta_{i,j}$ 
18      Compute task-adaptive loss values:  $\mathcal{L}_{\phi'_{i,j}}(t_{i,j})$ 
19      Perform inner-loop update on base model:
20       $\theta_{i,j+1} = \theta_{i,j} - \alpha \nabla_{\theta_{i,j}} \mathcal{L}_{\phi'_{i,j}}(t_{i,j})$ 
21     end
22   end
23   Perform outer-loop update on base model and both
24   meta-learners:
25    $(\theta, \phi, \psi) \leftarrow (\theta, \phi, \psi) - \beta \nabla_{(\theta, \phi, \psi)} \sum_{\mathcal{T}_i} \mathcal{L}(\mathcal{D}_i^Q; \theta_{i,J})$ 
26 end

```

BetterNet-HTTPS [49] dataset is another HTTPS traffic dataset, including the network traces during crawling top 779 HTTPS websites via both Chrome and Firefox browsers.

We adhere strictly to the labeling scheme established in [5], [10], [47]. For *CTU* and *BetterNet-HTTPS* datasets, we retain only those encrypted sessions with Server Name Indication (SNI) extension, and we use the processed hostnames (with numbers and special characters removed) as the label of each encrypted session. For *ISCX-VPN*, *ISCX-Tor*, *5GAD*, and *Google-Home* datasets, we assign labels based on the filenames. However, within the browsing label category of *ISCX-VPN* and *ISCX-Tor* datasets, there are traffic generated when performing other activities such as streaming and VoIP. To avoid ambiguity, We exclude such data in line with [10].

2) **Implementation:** METAROCKETC is primarily developed in C++ (GCC version 11.4.0) and Python (version 3.7.10). To promote transparency and repeatability, we have made our datasets and code public.¹

To replicate real-world network conditions, we utilize tcpplay (version 3.4.4) to playback these experimental traffic

datasets on specified interface. In highly efficient packet analyzer, we leverage PcapPlusPlus (version 22.05), a versatile C++ library with user-friendly APIs to perform real-time traffic analysis and feature extraction, including capturing packets by libpcap functionality, splitting packets by connections and parsing per-packet features within the IP layer. In the following three modules, we leverage Pytorch [50] (version 1.4.0) to generate multivariate time series, conduct random convolution kernel transformations, and perform task-adaptive classifications for various ETC tasks.

3) **Setup:** All experiments are simulated on a DELL server equipped with a 24-core Intel Xeon 6240R CPU @2.40GHz, Ubuntu 20.04, 64 GB memory, and 2×NVIDIA Quadro RTX 5000 GPUs. To ensure the robustness and generality, the experiments are replicated 10 times to calculate the average and standard variance of evaluation metrics. In each experiment, we utilize *ISCX-Tor* dataset to establish a common prior knowledge base, and then fine-tune the model respectively on the remaining datasets to assess the generalization and transferability from both across-task and few-shot perspectives.

In the across-task scenario, we evaluate METAROCKETC in both closed-world and open-world settings. In the closed-world setting, we test METAROCKETC and baseline models when transferring to different ETC tasks under varying label rates (i.e., 20%, 40%, and 60%). In the open-world setting, we employ *BetterNet-HTTPS* dataset with many classes to distinguish encrypted sessions between monitored and unmonitored classes, as outlined in [19].

In the few-shot scenario, all models are evaluated under 5-way 1-shot and 5-way 5-shot settings. Specifically, for baseline models tailored to specific ETC tasks using ML or DL algorithms, we adapt them into the MAML framework. As for generic ETC solutions using adversarial domain adaptation or contrastive learning, we follow the original solutions to acquire common knowledge and subject them to meta-testing.

In this manner, we fairly evaluate the performances of METAROCKETC and all baseline models in both across-task and few-shot scenarios.

B. Baseline Models

To thoroughly evaluate the generalization and transferability of METAROCKETC, we select six state-of-the-art methods as baseline models for comparisons. Among these, four are tailored for specific ETC tasks (e.g., application classification and website fingerprinting), utilizing ML or DL models. The other two are generic ETC solutions that employ adversarial domain adaptation and contrastive learning for fast task adaptation. It is notable that most of the selected methods are open-source and we directly use the associated open-source projects. For those that are not open-source, we have diligently reproduced them, adhering to the hyper-parameter settings detailed in the original papers. We fine-tune all baseline models to achieve their best performance on our experimental datasets. Below is the brief overview of all baseline models.

¹<https://github.com/MetaRocketC/MeTaRocketC>

ETC-PS [10] is a generalized encrypted traffic classification method leveraging path signature features. It introduces multiple transformations to enrich packet length sequences and model them as traffic paths. Then ETC-PS extracts multi-scale path signatures and employ traditional ML models to achieve efficient classification. We reproduce ETC-PS based on the original paper.

GRAPHDAPP [19] models bidirectional packet length sequences as Traffic Interaction Graphs (TIGs) and turns encrypted traffic classification into a graph classification problem, where multi-layer perceptions are employed to build a GNN-based classifier to identify decentralized applications through encrypted traffic analysis. We reproduce GRAPHDAPP based on the original paper.

FLOWPIC [51] transforms network flows into images. Specifically, the records of each packet (i.e., packet size, arrival time) are mapped into 2D-histograms to be square images. Then LeNet-5 style CNN is utilized to build encrypted traffic classifiers. The open-source implementation of FLOWPIC is available in [52].

FS-NET [18] is an end-to-end model for encrypted traffic classification. It takes multi-layer bidirectional Gated Recurrent Unit (bi-GRU) encoder and decoder to learn the representation of the flow sequence and reconstructs the original sequence respectively. Both the encoder and decoder output are combined for classification. The open-source implementation of FS-NET is available in [53].

AF [5] is a transfer learning-based website fingerprinting technique that leverages adversarial domain adaptation over few encrypted traffic. Specifically, it adopts a domain adversarial network architecture where a feature extractor and a domain discriminator engage in a minimax game. The feature extractor is attached with traditional machine learning classifiers to facilitate practical website fingerprinting. The open-source implementation of FS-NET is available in [54].

PACREP [6] is a novel framework to learn generic packet representation for various traffic classification tasks. PACREP is optimized by contrastive loss from both positive and negative samples to tackle both semantic and byte information and learn the packet representations that are commonly effective in different traffic classification tasks. The open-source implementation of FS-NET is available in [55].

C. Across-Task Analysis

1) *Closed-World Evaluation:* In this subsection, we assess the performance of METAROCKETC in the closed-world setting under an across-task scenario. In the closed-world setting, the classes of all encrypted sessions to be classified are monitored and encrypted traffic classification is treated as a general multi-class classification problem.

To validate the efficacy of METAROCKETC in facilitating fast adaptation to various ETC tasks, we compare METAROCKETC with other baseline models using three datasets with varying label rates (i.e., 20%, 40%, and 60%). In this way, we can evaluate the performance of ETC solutions with different amounts of available labeled data, enabling a

comprehensive evaluations of scalability, cost management, and practicality within real-world applications. Specifically, we utilize *ISCX-Tor* dataset to train the meta-classifier to acquire common prior knowledge, and then fine-tune it on specific datasets. The experimental results are summarized in Table V.

We can make the following observations.

- 1) *Superior Performance of Generic ETC Solutions:* METAROCKETC, AF, and PACREP outperforms other baseline models in most cases. These results align with our expectations, considering that METAROCKETC, AF, and PACREP are generic ETC solutions not confined to specific tasks. They leverage common knowledge across diverse ETC tasks, consistently exhibiting superior adaptability when transferring to unknown ETC tasks with limited training data.
- 2) *Technique Efficacy in Generic ETC Solutions:* Notably, METAROCKETC, employing meta-learning, AF, utilizing adversarial domain adaptation, and PACREP, relying on contrastive learning, are basically the top three models. These models share a common strategy of learning a generic feature encoder. METAROCKETC and AF share structural similarities, where AF employs convolutional neural networks, while METAROCKETC leverages random convolution transformations for feature encoding. Moreover, both METAROCKETC and PACREP acquire generic representations by designing specific loss functions. METAROCKETC take merits from AF, PACREP and advanced time series technique (ROCKET and its variants), achieving the best performances.
- 3) *Exceptional Performance of Task-Specific GraphDAPP:* Among the baseline models designed for specific ETC tasks, GRAPHDAPP is a special case, which constructs Traffic Interaction Graphs (TIGs) with packet length sequences and employs GNN for decentralized application classification. GRAPHDAPP surpasses generic ETC solutions in several instances (e.g., 60% label rate on *ISCX-VPN* dataset and 20% label rate on *BetterNet-HTTPS* dataset). This exceptional performance is attributed to the discriminative and generic nature of the TIG representation across diverse ETC tasks.
- 4) *Performance Variations Across Datasets:* It is noteworthy that on *BetterNet-HTTPS* dataset with 20% label rate, ETC-PS outperforms METAROCKETC. The reason for the difference may be the larger number of classes within the *BetterNet-HTTPS* dataset (950 classes) compared to other datasets (e.g., 12 and 59 classes). This higher class diversity may favor the path signature representations of ETC-PS for this particular dataset. However, as the label rate increase, these generic ETC solutions outperform ETC-PS.
- 5) *Performance Scaling with Label Rate:* As the label rate increases, the performances of METAROCKETC and other baseline models improve, where FLOWPIC and FS-NET exhibit the most significant improvements. The LeNet-5 style CNN of FLOWPIC and bi-GRU style encoder and decoder of FS-NET require substantial amounts of labeled data. As such, their practicality in

TABLE V
EFFECTIVENESS EVALUATION ON ISCX-VPN, CTU AND BETTERNET-HTTPS DATASETS

Dataset	Label Rate	Metric	ETC-PS	GRAPHDAPP	FlowPic	FS-NET	AF	PACREP	METAROCKETC
ISCX-VPN	20%	AC	78.02±0.65%	79.92±0.68%	49.98±1.25%	72.20±0.92%	81.82±1.03%	81.39±0.69%	85.67±0.55%
		PR	76.58±0.72%	78.19±0.54%	52.17±1.31%	72.67±0.50%	83.57±0.99%	80.84±0.57%	82.99±0.51%
		RC	78.74±0.48%	79.48±0.71%	49.98±0.98%	72.01±0.79%	81.82±1.15%	82.71±0.66%	84.47±0.57%
		F1	75.77±0.41%	78.11±0.52%	47.60±1.12%	70.27±0.49%	80.43±0.98%	78.48±0.59%	82.82±0.52%
	40%	AC	84.26±0.57%	86.56±0.59%	79.82±1.10%	81.32±0.72%	87.54±0.92%	87.45±0.77%	89.67±0.41%
		PR	83.67±0.38%	88.22±0.47%	78.24±0.91%	81.32±0.58%	86.11±0.87%	86.46±0.69%	87.59±0.32%
		RC	83.61±0.45%	85.97±0.42%	79.44±0.67%	81.32±0.62%	87.02±0.91%	86.43±0.62%	89.02±0.25%
		F1	82.88±0.39%	85.72±0.39%	77.68±0.74%	81.18±0.61%	84.55±0.72%	85.09±0.54%	87.22±0.37%
	60%	AC	87.87±0.50%	91.44±0.45%	89.65±0.97%	85.75±0.74%	91.89±0.88%	91.40±0.84%	93.33±0.29%
		PR	87.90±0.41%	92.62±0.38%	90.11±0.98%	85.75±0.52%	91.66±0.81%	91.64±0.53%	92.98±0.22%
		RC	87.89±0.43%	91.38±0.33%	88.12±0.73%	85.75±0.44%	90.52±0.73%	90.79±0.45%	93.18±0.29%
		F1	86.22±0.37%	90.67±0.35%	87.64±0.88%	85.50±0.49%	88.78±0.82%	89.04±0.46%	92.81±0.21%
CTU	20%	AC	67.27±0.72%	69.14±0.73%	48.89±1.37%	55.33±0.81%	69.13±0.88%	69.35±0.83%	75.14±0.67%
		PR	67.25±0.70%	67.93±0.68%	49.99±1.22%	59.38±0.74%	70.66±0.72%	71.29±0.64%	76.87±0.62%
		RC	67.22±0.54%	67.97±0.66%	47.24±0.91%	55.33±0.63%	68.71±0.87%	70.69±0.57%	75.09±0.56%
		F1	66.89±0.58%	66.17±0.61%	47.06±0.95%	55.34±0.61%	68.24±0.95%	70.23±0.58%	74.56±0.61%
	40%	AC	76.24±0.69%	82.21±0.59%	74.58±1.26%	72.33±0.79%	83.23±0.72%	84.02±0.75%	86.15±0.48%
		PR	75.82±0.51%	81.88±0.50%	72.91±0.89%	73.24±0.51%	83.95±0.70%	83.19±0.61%	84.42±0.41%
		RC	75.46±0.34%	81.88±0.46%	73.48±0.96%	72.34±0.33%	84.27±0.64%	83.22±0.64%	85.09±0.29%
		F1	75.13±0.47%	81.76±0.49%	71.42±0.95%	71.67±0.48%	81.43±0.81%	81.15±0.43%	84.14±0.38%
	60%	AC	84.19±0.66%	90.67±0.47%	85.57±1.11%	80.44±0.71%	90.99±0.59%	89.45±0.68%	92.00±0.34%
		PR	83.87±0.44%	88.95±0.39%	85.92±0.78%	81.69±0.64%	90.16±0.55%	90.04±0.48%	90.60±0.21%
		RC	83.51±0.30%	88.82±0.34%	85.92±0.92%	80.04±0.30%	90.22±0.57%	89.64±0.39%	90.12±0.33%
		F1	83.26±0.31%	87.69±0.33%	84.77±0.81%	79.67±0.59%	89.21±0.60%	88.91±0.47%	89.73±0.25%
BetterNet-HTTPS	20%	AC	80.17±0.71%	79.82±0.73%	53.19±0.95%	76.35±0.87%	80.44±1.07%	78.33±0.84%	80.01±0.61%
		PR	81.55±0.65%	81.79±0.67%	52.94±0.86%	77.72±0.66%	80.05±0.86%	79.74±0.62%	81.52±0.52%
		RC	80.91±0.57%	81.06±0.62%	53.90±0.96%	77.46±0.54%	79.11±0.89%	79.42±0.51%	79.49±0.59%
		F1	80.16±0.53%	80.04±0.61%	51.05±0.89%	76.32±0.57%	80.75±0.94%	79.33±0.56%	79.40±0.48%
	40%	AC	88.19±0.59%	89.27±0.54%	84.99±0.82%	85.17±0.73%	89.65±0.72%	89.38±0.71%	90.05±0.44%
		PR	88.96±0.43%	89.72±0.59%	86.01±0.99%	86.42±0.57%	89.28±0.64%	89.46±0.56%	89.18±0.37%
		RC	88.34±0.37%	89.91±0.43%	85.76±0.81%	86.15±0.40%	89.32±0.65%	88.23±0.55%	89.99±0.28%
		F1	88.12±0.38%	88.66±0.33%	84.49±0.88%	85.11±0.48%	89.48±0.57%	89.13±0.59%	89.11±0.31%
	60%	AC	91.96±0.51%	94.52±0.39%	93.56±0.75%	89.51±0.69%	95.13±0.63%	93.47±0.72%	96.67±0.30%
		PR	91.72±0.39%	93.66±0.34%	94.44±0.79%	89.11±0.31%	95.13±0.63%	94.13±0.42%	94.83±0.24%
		RC	91.58±0.33%	93.81±0.27%	93.88±0.59%	89.05±0.44%	95.48±0.49%	94.16±0.45%	96.27±0.35%
		F1	91.40±0.38%	93.07±0.25%	92.86±0.66%	89.01±0.37%	93.99±0.48%	94.15±0.37%	94.66±0.21%
5GAD	20%	AC	76.92±0.79%	76.79±0.82%	49.72±1.88%	70.53±0.73%	77.59±1.82%	77.12±0.92%	82.03±0.78%
		PR	76.99±0.82%	79.96±0.79%	51.35±1.67%	73.56±0.85%	81.08±1.67%	77.98±1.05%	85.38±0.52%
		RC	76.52±0.64%	76.62±0.80%	48.65±1.91%	70.67±0.72%	77.21±1.49%	75.96±0.85%	82.10±0.78%
		F1	74.81±0.66%	74.28±0.86%	46.16±1.94%	66.67±0.82%	75.04±1.87%	74.55±0.94%	76.40±1.52%
	40%	AC	86.33±0.98%	87.03±0.95%	76.13±1.09%	79.54±0.96%	87.68±0.98%	89.52±0.63%	91.67±1.19%
		PR	87.67±0.82%	89.97±0.81%	77.28±0.97%	81.44±0.72%	90.82±0.91%	90.67±0.76%	92.70±1.02%
		RC	85.12±0.67%	87.54±0.79%	77.51±0.91%	80.48±0.56%	89.65±0.95%	88.95±0.89%	91.82±1.19%
		F1	85.03±0.72%	85.32±0.98%	75.08±0.94%	78.13±0.67%	87.18±1.01%	86.52±0.95%	90.73±1.75%
	60%	AC	92.77±0.63%	96.05±0.77%	92.33±0.78%	87.21±0.64%	96.82±0.74%	96.33±0.97%	97.31±0.46%
		PR	92.52±0.75%	96.77±0.68%	93.64±0.83%	85.45±0.57%	96.95±0.69%	96.49±0.64%	97.67±0.38%
		RC	90.02±0.59%	95.24±0.71%	91.24±0.66%	86.34±0.62%	96.80±0.77%	95.59±0.78%	97.27±0.46%
		F1	89.15±0.68%	94.81±0.85%	90.05±0.72%	83.33±0.51%	96.28±0.81%	95.57±0.72%	96.81±0.48%
Google-Home	20%	AC	74.35±0.72%	75.07±0.64%	39.24±1.71%	65.33±1.14%	75.42±1.28%	76.15±1.03%	79.42±0.45%
		PR	77.21±0.93%	76.53±1.32%	42.73±1.80%	67.58±1.03%	78.56±0.97%	78.56±0.96%	81.45±1.67%
		RC	72.98±0.84%	74.26±0.75%	40.19±1.28%	65.12±0.98%	76.39±1.07%	76.11±0.92%	76.47±0.45%
		F1	70.99±0.95%	72.20±0.92%	35.74±1.34%	62.44±0.98%	72.88±0.92%	73.98±0.83%	74.01±0.66%
	40%	AC	86.45±0.95%	87.04±0.91%	74.07±0.92%	83.27±0.79%	88.55±1.12%	88.23±0.77%	89.12±0.92%
		PR	88.31±1.12%	90.21±1.04%	74.49±1.33%	84.97±0.72%	88.82±1.57%	89.63±0.67%	91.45±1.48%
		RC	88.10±0.99%	88.98±1.14%	73.75±1.03%	83.03±0.65%	87.50±0.90%	88.65±0.62%	88.33±0.92%
		F1	86.52±1.05%	85.75±1.27%	71.55±0.96%	81.99±0.68%	86.63±1.33%	86.36±0.71%	86.89±1.12%
	60%	AC	92.12±0.65%	98.76±0.64%	94.88±0.76%	89.27±0.74%	99.22±0.59%	99.05±0.65%	99.28±0.41%
		PR	95.95±0.79%	99.03±0.53%	97.21±0.62%	90.51±0.68%	99.25±0.51%	99.64±0.62%	99.36±0.36%
		RC	94.10±0.82%	99.31±0.49%	96.24±0.71%	88.10±0.61%	99.28±0.48%	99.48±0.71%	99.28±0.41%
		F1	91.88±0.65%	97.64±0.66%	95.05±0.69%	85.43±0.57%	98.97±0.62%	98.91±0.75%	99.01±0.47%

We highlight the best in ●, the second best in ●, and the third best in ●.

real-world environments may be limited. Furthermore, even with enough labeled data available (i.e., 60% label rate), METAROCKETC and the two other generic ETC solutions still outperform these task-specific models. This underscores the idea that the common knowledge acquired does not affect the adaptability of models, but enhances their ability to tackle novel ETC tasks.

2) *Open-World Evaluation*: In this subsection, we evaluate the performance of METAROCKETC in the open-world setting under an across-task scenario. In a more realistic way, the goal of METAROCKETC is to distinguish between encrypted sessions from monitored and unmonitored classes. In this

way, encrypted traffic classification is treated as a binary classification problem.

Specifically, we use *BetterNet-HTTPS* dataset for open-world evaluation. The training part of *BetterNet-HTTPS* dataset consists of 34,200 monitored encrypted sessions from 950 monitored classes and 4,750 unmonitored encrypted sessions from 500 unmonitored classes). The testing part consists of 8,550 monitored encrypted sessions from 950 monitored classes) and 17,641 unmonitored encrypted sessions from 819 unmonitored classes). Notably, the unmonitored encrypted sessions in the training set are totally different from those in testing set.

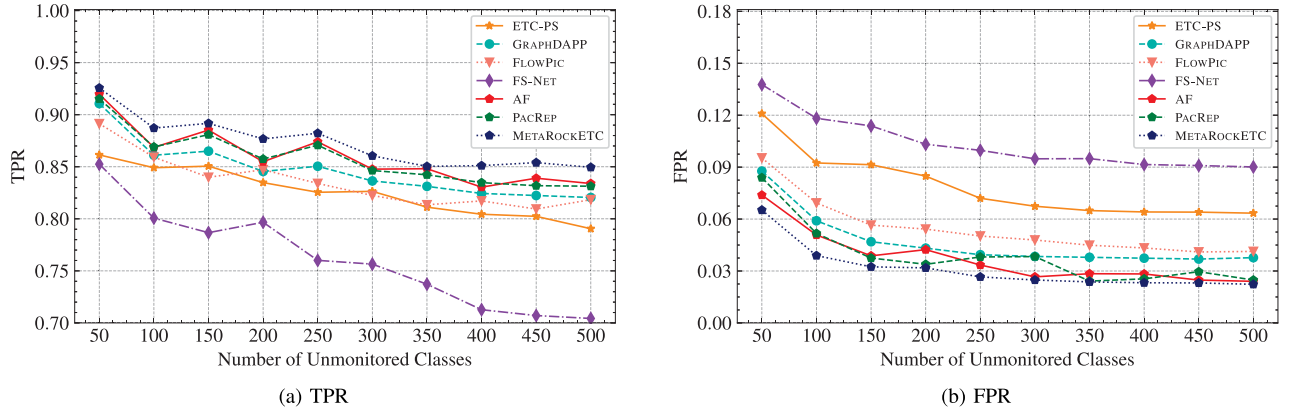


Fig. 4. The impact of the number of unmonitored classes in training set on TPR and FPR.

TABLE VI
5-WAY 1-SHOT AND 5-WAY 5-SHOT CLASSIFICATION ACCURACY ON THE FOUR EXPERIMENTAL DATASETS

Dataset	N -way k -shot	GRAPHDAPP	FLOWPIC	FS-NET	AF	PACREP	METAROCKETC
<i>ISCX-VPN</i>	5-way 1-shot	63.58 $\pm$ 0.31%	60.08 $\pm$ 0.52%	59.82 $\pm$ 0.46%	63.82 $\pm$ 0.40%	63.64 $\pm$ 0.35%	64.67 $\pm$ 0.31%
	5-way 5-shot	71.84 $\pm$ 0.29%	71.21 $\pm$ 0.44%	70.17 $\pm$ 0.38%	72.99 $\pm$ 0.32%	72.67 $\pm$ 0.47%	73.99 $\pm$ 0.25%
<i>CTU</i>	5-way 1-shot	52.51 $\pm$ 0.40%	52.45 $\pm$ 0.67%	52.87 $\pm$ 0.49%	52.56 $\pm$ 0.42%	52.43 $\pm$ 0.58%	52.41 $\pm$ 0.34%
	5-way 5-shot	58.63 $\pm$ 0.34%	56.88 $\pm$ 0.48%	55.59 $\pm$ 0.37%	58.59 $\pm$ 0.46%	58.67 $\pm$ 0.61%	58.70 $\pm$ 0.31%
<i>BetterNet-HTTPS</i>	5-way 1-shot	70.57 $\pm$ 0.28%	67.52 $\pm$ 0.45%	64.72 $\pm$ 0.34%	71.02 $\pm$ 0.31%	70.53 $\pm$ 0.59%	71.33 $\pm$ 0.30%
	5-way 5-shot	79.84 $\pm$ 0.21%	76.24 $\pm$ 0.36%	75.69 $\pm$ 0.28%	80.78 $\pm$ 0.28%	80.54 $\pm$ 0.36%	82.92 $\pm$ 0.24%
<i>Google-Home</i>	5-way 1-shot	48.27 $\pm$ 0.61%	45.16 $\pm$ 0.59%	46.67 $\pm$ 0.60%	47.50 $\pm$ 0.61%	47.31 $\pm$ 0.68%	47.52 $\pm$ 0.58%
	5-way 5-shot	56.33 $\pm$ 0.59%	54.72 $\pm$ 0.52%	55.40 $\pm$ 0.57%	56.84 $\pm$ 0.48%	56.59 $\pm$ 0.53%	57.15 $\pm$ 0.42%

We highlight the best in ●, the second best in ●, and the third best in ●.

Here we vary the number of unmonitored classes in the training set from 50 to 500 to simulate the realistic scenario and investigate the impact of more complex training data on the model performance. As illustrated in Fig. 4, both the True Positive Rate (TPR) and False Positive Rate (FPR) decrease as the number of unmonitored classes in the training set increases for METAROCKETC and other baseline models, where METAROCKETC still performs best on both TPR and FPR for 500 unmonitored classes, achieving an impressive TPR of 84.95% and a low FPR of 2.23%. When the number of unmonitored classes in the training set reaches 350, the TPR and FPR of METAROCKETC stabilize as 85.04% and 2.37% respectively.

Overall, these experimental results demonstrate the robustness and effectiveness of METAROCKETC in the open-world scenario, even when faced with an increasing number of unmonitored classes during the training phase.

D. Few-Shot Analysis

For comprehensive evaluations in a few-shot scenario, we compare METAROCKETC with five gradient-based baseline models (i.e., GRAPHDAPP, FLOWPIC, FS-NET, AF and PACREP) integrated with MAML++ under two typical few-shot settings: 5-way 1-shot and 5-way 5-shot classification. Since there are only two classes in *5GAD* dataset, we do not use it for few-shot analysis. Given that the datasets are organized into distinct tasks and balanced in N -way k -shot settings, we utilize accuracy as the metric to present the experimental results.

As presented in Table VI, METAROCKETC achieves average improvements of approximately 1.33% and 2.18% on accuracy in 5-way 1-shot and 5-way 5-shot settings, respectively, across four datasets. As with the experimental results of across-task analysis, generic ETC solutions (i.e., AF and PACREP) outperform other baseline models focusing on specific ETC tasks. Among these baseline models, GRAPHDAPP achieves the best performance, which further prove that TIG representation is effective in encrypted traffic classification across tasks.

Additionally, though METAROCKETC does achieve the best performances in most cases within the few-shot regime, the improvements are not significant as expected. The reason maybe that we integrate all baseline models with MAML++ for fair comparisons and the meta-learning techniques effectively fine-tune all baseline models, thus exhibiting competitive performances to METAROCKETC. However, it is noteworthy that METAROCKETC consistently outperforms all baseline models on all datasets in the 5-way 5-shot setting.

Overall, these results not only underscore the potential of meta-learning in developing practical ETC solutions, but also emphasize the superiority of METAROCKETC within the context of few-shot ETC tasks, indicating that it is a promising and practical solution for tackling the challenges posed by complex network environments.

E. Efficiency Evaluation

1) *Adaptation Efficiency*: Here we take *ISCX-VPN* dataset (20%) as an example to further demonstrate the fast adaptation performance of METAROCKETC. As illustrated in Fig. 5,

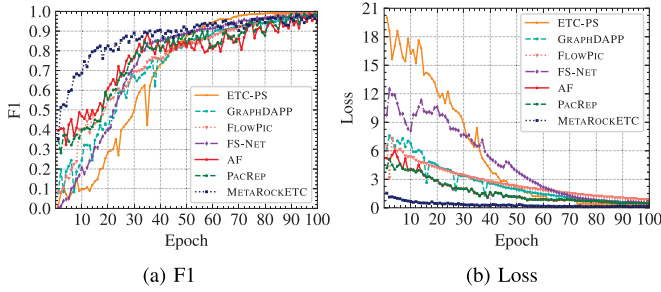


Fig. 5. F1 score and loss evaluations of METAROCKETC and other baseline models on *ISCX-VPN* dataset (20%) during fine-tuning phase.

TABLE VII
TIME OVERHEAD EVALUATION ON 5GAD DATASET

Phase	AF	PACREP	METAROCKETC
Feature Extraction	209.37 s	> 10 h	209.37 s
Model Fine-tuning	319.35 s	573.42 s	170.65 s
Model Execution	11.17 s	18.29 s	5.60 s

METAROCKETC can rapidly achieve satisfactory F1 score and loss with a fewer number of epochs compared with other baseline models. This nature enables METAROCKETC to be practical to handle different ETC tasks and evolving network environments. AF and PACREP integrating the common knowledge of previous ETC task also exhibit fast adaptation ability to *ISCX-VPN* dataset. Moreover, we can see that when label rate is set to 20%, the other baseline models more or less over-fit to the training set and exhibit poor performances on the testing set. While METAROCKETC still achieve the best generalization performance on the testing set as shown in Table V.

The experimental results suggest that METAROCKETC effectively incorporates the common knowledge of different ETC tasks, exhibiting exceptional transferability to adapt to new ETC tasks. The reason behind these may be due to the discriminatory patterns extracted by random convolution transformations or the task-adaptive loss function, which will be comprehensively discussed in Section V-F. From a practical perspective, METAROCKETC effectively bridges the gap between laboratory experimentation and the real-world environments, providing a practical solution for adapting to diverse ETC tasks.

2) *Time Overhead*: Considering the practical deployment within real-world scenarios, we conduct a comprehensive evaluation of the time overhead associated with METAROCKETC and two practical ETC solutions (i.e., AF and PACREP) in different phases including feature extraction, model fine-tuning and model execution. Specifically, we take *5GAD* dataset for evaluations, where 50% for model fine-tuning, and 50% for model execution. The time overheads for all three methods are presented in Table VII.

In a nutshell, METAROCKETC is significantly faster than both AF and PACREP across all phases. Notably that AF is originally designed for well-processed website fingerprinting datasets, we use the same preprocessing code as METAROCKETC to process *5GAD* dataset. Therefore the

feature extraction times for AF and METAROCKETC are the same.

Though METAROCKETC conducts plenty of convolution transformations on the packet length sequences, all the transformations are executed in a parallel and independent manner. Furthermore, the convolution kernels are deterministically generated without training. Additionally, METAROCKETC leverages 4-layer MLP as the base model for feature encoding, which simple structure does not require much computation overhead. These three factors collectively render METAROCKETC highly efficient and applicable.

Comparatively, AF similarly relies on simple features and a model structure similar to METAROCKETC. It utilizes convolutional neural network as the feature extractor, which require training. As for PACREP, it extracts both encrypted and unencrypted data as text and hexadecimal bytes utilizing the scapy library, which is time-consuming compared to METAROCKETC and AF. It has been demonstrated that packet lengths are effective and generic across different ETC tasks by METAROCKETC and GRAPHDAPP.

It is worth noting that PACREP are implemented based on its own open-source code. The efficiencies of the components (e.g., feature extractor) in PACREP can be further optimized with PcapPlusPlus. However, even taking this factor into account, METAROCKETC still demonstrates superior efficiency in comparison with the two baselines.

F. Ablation Study

To further substantiate the efficacy of the innovative components, we conduct a careful ablation analysis on the key components of METAROCKETC. Specifically, we compare the full METAROCKETC with its three variants:

- *w/o MV*: w/o MV solely utilizes the original bidirectional packet length sequences as univariate time series for subsequent analysis. Consequently, the random channel sampling and summation operation in multivariate version of METAROCKETC are inapplicable.
- *w/o TL*: w/o TL removes the two meta-learners that serve as the task-adaptive loss function in inner-loop optimization, instead employing a general cross-entropy loss function in accordance with the vanilla MAML framework.
- *w/o FRZ*: w/o FRZ does not incorporate the transfer learning technique of freezing. Instead, it updates all parameters during fine-tuning process on specific tasks.

To ensure fair comparisons, METAROCKETC and its variants are fine-tuned for an equal number of epochs. We present the experimental results of METAROCKETC and its variants on three datasets with 20% label rate in Table VIII. As observed, METAROCKETC achieves the best classification performance with remarkable stability across all tasks. The respective average decreases of 1.74%, 5.01% and 13.29% in macro-average F1 score for w/o MV, w/o TL and w/o FRZ on three datasets indicate that all our proposed improvements are both practical and effective for various ETC tasks. Among these w/o MV exhibits the least performance

TABLE VIII
ABLATION STUDY OF METAROCKETC ON THE FIVE EXPERIMENTAL DATASETS (20%)

Dataset	Metric	w/o MV	w/o TL	w/o FRZ	METAROCKETC
<i>ISCX-VPN</i>	AC	82.91±0.61%	79.07±0.65%	71.51±0.91%	85.67±0.55%
	PR	81.68±0.65%	78.36±0.71%	70.95±0.83%	82.99±0.51%
	RC	82.27±0.70%	78.92±0.67%	71.52±0.89%	84.47±0.57%
	F1	81.39±0.69%	77.94±0.74%	70.11±0.92%	82.82±0.52%
<i>CTU</i>	AC	74.04±0.74%	70.53±0.78%	62.64±0.97%	75.14±0.67%
	PR	72.92±0.71%	70.21±0.83%	61.03±0.90%	76.87±0.62%
	RC	73.85±0.74%	70.04±0.81%	62.71±0.95%	75.09±0.56%
	F1	72.89±0.75%	69.84±0.85%	60.92±0.99%	74.56±0.61%
<i>BetterNet-HTTPS</i>	AC	78.58±0.69%	75.87±0.75%	67.27±0.97%	80.01±0.61%
	PR	77.68±0.73%	74.91±0.81%	66.12±0.91%	81.52±0.52%
	RC	78.41±0.80%	75.24±0.79%	67.31±0.94%	79.49±0.59%
	F1	77.29±0.75%	73.98±0.86%	65.89±0.99%	79.40±0.48%
<i>5GAD</i>	AC	78.12±0.82%	76.56±0.98%	73.44±1.01%	82.03±0.78%
	PR	83.15±0.79%	70.99±0.90%	66.18±0.92%	85.38±0.52%
	RC	78.12±0.91%	76.62±1.12%	73.00±1.25%	82.10±0.78%
	F1	71.89±1.64%	68.97±0.86%	70.31±1.86%	76.40±1.52%
<i>Google-Home</i>	AC	75.64±0.57%	71.17±0.81%	77.06±0.79%	79.42±0.45%
	PR	73.59±0.74%	72.89±0.65%	77.98±0.91%	81.45±1.67%
	RC	75.88±0.66%	70.15±0.77%	74.21±0.73%	76.47±0.45%
	F1	74.15±0.61%	71.67±0.69%	74.45±0.67%	74.01±0.66%

decrease, suggesting that transforming bidirectional packet length sequences from univariate and multivariate time series results in a slight improvement. Introducing diversity can enrich encrypted traffic representations and enhance classification performance to some extent. Furthermore, by comparing METAROCKETC with w/o TL, we demonstrate that a simple fixed loss function cannot generalize well across diverse and distinct ETC tasks. Employing neural networks to fit a task-adaptive loss function can significantly improve the generalization and transferability. Lastly, we assess the efficacy of the freezing technique when adapting the base model to different ETC tasks. The performance of w/o FRZ declines considerably. One possible explanation is that, when adapting the base model to new tasks, the initial layers have learned common knowledge from known ETC tasks. Without adopting the freezing technique, the adaptation process becomes retraining, and the weights are compromised during subsequent training iterations. In this case, fitting new ETC tasks require a significantly larger number of training epochs, rendering this approach impractical for real-world scenarios. Overall, METAROCKETC surpasses all variants by integrating multivariate time series, task-adaptive loss function, and freezing mechanism to rapidly adapt to diverse ETC tasks and evolving network environments.

VI. DISCUSSION

METAROCKETC exhibits strong generalization and transferability to quickly adapt to different encrypted traffic classification tasks, yet there is still room for improvement. In this section, we discuss how to further optimize METAROCKETC in the following four aspects.

A. Diversity

To add diversity in METAROCKETC, we perform a series of transformations to generate multivariate time series and utilize multiple pooling operators to summarize the convolution

outputs. Experimental results demonstrate that diversity is the key to improve the discriminatory power of the extracted features. In future work, we can further increase the diversity by considering other per-packet features (e.g., inter-arrival time and protocols) and design more pooling operators to capture more comprehensive traffic patterns for classification. By leveraging meta-learning, significant reductions in resource consumption for parameter updates can be achieved, thus eliminating concerns regarding feature dimension explosion.

B. Efficiency

We have implemented a highly efficient packet analyzer in C++ and the remaining three modules in Python for METAROCKETC. Although our meta-learning framework significantly reduces the need for frequent retraining, recent advancements in Intelligent Data Plane (IDP) offer the potential to further promote the efficiency of METAROCKETC. IDP leverages programmable switches to provides essential hardware support, enabling the deployment of learning-based models directly onto the network data plane. Integrating METAROCKETC with an intelligent network data plane could facilitate real-time intelligent traffic analysis and improve the efficiency.

C. Scalability

Our aim is to develop a generic encrypted traffic classification framework which can acquire across-task knowledge and rapidly adapt to unseen tasks and evolving network environments. To achieve this, METAROCKETC employs simple yet effective features and follows a meta-learning framework, significantly reducing the burden of frequent retraining. However, one notable inadequacy is its reliance on supervised learning, which may not accurately reflect real-world scenarios, where the exact number of classes is often unknown. To better align with the realistic scenario, we can incorporate unsupervised meta-learning to further decrease data dependency and

enhance METAROCKETC's adaptability to complex and time-evolving environments and diverse downstream tasks.

D. Practicality

METAROCKETC provides a practical solution adaptable to various ETC tasks. Especially, we highlight its strength in network security analysis against dynamic threat landscapes. The meta-learning ability of METAROCKETC facilitates timely identification of emerging threats over limited traffic data, allowing for rapid response. For example, IoT devices without integrated security, are susceptible to cyber attacks. METAROCKETC can efficiently monitor the network activities of IoT devices via encrypted traffic analysis, enabling prompt detection and isolation of suspicious/compromised devices and effectively prevents threat activities from the outset. For future work, we can integrate continual learning to develop METAROCKETC adaptively. We also see potential in further refining its functionalities across different ETC tasks (e.g., real-time alert for threat detection) to enhance its practical applications.

VII. CONCLUSION

In this paper, we present METAROCKETC, an innovative encrypted traffic classification framework that leverages protocol-agnostic time series features and task-adaptive meta-learning techniques to achieve better generalization across various ETC tasks and rapidly adapt to unknown tasks and evolving environments. By modeling packet length sequences as multivariate time series and performing random convolution kernel transformations, METAROCKETC effectively captures the discriminatory behavioral patterns of encrypted traffic for subsequent classification. We integrate the backbone network with two meta-learners to learn task-adaptive loss functions in the MAML manner, enabling task-adaptive learning in dynamic network environments. Extensive experimental results have demonstrated the superiority of METAROCKETC in both across-task and few-shot scenarios. METAROCKETC offers a more practical and robust solution for encrypted traffic classification in real-world network environments.

For future research, we suggest incorporating more advanced time series analysis techniques and exploring alternative meta-learning approaches to further enhance the generalization performance of this framework. Additionally, investigating the robustness of the proposed method against adversarial attacks and integrating it with real-time network monitoring systems could provide valuable research avenues.

REFERENCES

- [1] Google. "Google transparency report." Accessed: May 31, 2023. [Online]. Available: <https://transparencyreport.google.com/https/overview>
- [2] T.-D. Pham, T.-L. Ho, T. Truong-Huu, T.-D. Cao, and H.-L. Truong, "MAppGraph: Mobile-App classification on encrypted network traffic using deep graph convolution neural networks," in *Proc. 37th Annu. Comput. Secur. Appl. Conf.*, 2021, pp. 1025–1038.
- [3] Z. Fu et al., "Encrypted malware traffic detection via graph-based network analysis," in *Proc. 25th Int. Symp. Res. Attacks, Intrusions Def.*, 2022, pp. 495–509.
- [4] A. Sivanathan et al., "Classifying IoT devices in smart environments using network traffic characteristics," *IEEE Trans. Mobile Comput.*, vol. 18, no. 8, pp. 1745–1759, Aug. 2019.
- [5] C. Wang, J. Dani, X. Li, X. Jia, and B. Wang, "Adaptive fingerprinting: Website fingerprinting over few encrypted traffic," in *Proc. 11th ACM Conf. Data Appl. Secur. Privacy*, 2021, pp. 149–160.
- [6] X. Meng, Y. Wang, R. Ma, H. Luo, X. Li, and Y. Zhang, "Packet representation learning for traffic classification," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Min.*, 2022, pp. 3546–3554.
- [7] K. Kostas, M. Just, and M. A. Lones, "IoTDevID: A behavior-based device identification method for the IoT," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 23741–23749, Dec. 2022.
- [8] B. Anderson, S. Paul, and D. McGrew, "Deciphering malware's use of TLS (without decryption)," 2016, arXiv:1607.01639.
- [9] J. Zhao, Q. Li, S. Liu, Y. Yang, and Y. Hong, "Towards traffic supervision in 6G: A graph neural network-based encrypted malicious traffic detection method," *SCIENTIA SINICA Inf.*, vol. 52, no. 2, pp. 270–286, 2022. [Online]. Available: <http://www.sciengine.com/publisher/ScienceChinaPress/journal/SCIENTIASINICAInformationis/52/2/10.1360/SSI-2021-0280>
- [10] S.-J. Xu, G.-G. Geng, X.-B. Jin, D.-J. Liu, and J. Weng, "Seeing traffic paths: Encrypted traffic classification with path signature features," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2166–2181, 2022.
- [11] A. Dempster, F. Petitjean, and G. I. Webb, "ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels," *Data Min. Knowl. Discov.*, vol. 34, no. 5, pp. 1454–1495, 2020.
- [12] A. Dempster, D. F. Schmidt, and G. I. Webb, "Minirocket: A very fast (almost) deterministic transform for time series classification," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Min.*, 2021, pp. 248–257.
- [13] C. W. Tan, A. Dempster, C. Bergmeir, and G. I. Webb, "MultiRocket: Multiple pooling operators and transformations for fast and effective time series classification," 2022, arXiv:2102.00457.
- [14] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," 2017, arXiv:1703.03400.
- [15] A. Antoniou, H. Edwards, and A. Storkey, "How to train your MAML," 2019, arXiv:1810.09502.
- [16] S. Baik, M. Choi, J. Choi, H. Kim, and K. M. Lee, "Learning to learn task-adaptive hyperparameters for few-shot learning," *IEEE Trans. Anal. Mach. Intell.*, early access, Mar. 24, 2023, doi: [10.1109/TPAMI.2023.3261387](https://doi.org/10.1109/TPAMI.2023.3261387).
- [17] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, 2016, pp. 439–454.
- [18] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun. INFOCOM*, 2019, pp. 1171–1179.
- [19] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [20] B. Anderson and D. McGrew, "Identifying encrypted malware traffic with contextual flow data," in *Proc. ACM Workshop Artif. Intell. Secur.*, 2016, pp. 35–46. [Online]. Available: <https://dl.acm.org/doi/10.1145/2996758.2996768>
- [21] B. Anderson and D. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2017, pp. 1723–1732. [Online]. Available: <https://dl.acm.org/doi/10.1145/3097983.3098163>
- [22] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 3431–3446. [Online]. Available: <http://arxiv.org/abs/2106.14707>
- [23] C. Fu, Q. Li, M. Shen, and K. Xu, "Frequency domain feature based robust malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 452–467, Feb. 2023.
- [24] C. Duan, H. Gao, G. Song, J. Yang, and Z. Wang, "ByteIoT: A practical IoT device identification system based on packet length distribution," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 2, pp. 1717–1728, Jun. 2022.
- [25] J. Zhao, Q. Li, J. Sun, M. Dong, K. Ota, and M. Shen, "Efficient IoT device identification via network behavior analysis based on time series dictionary," *IEEE Internet Things J.*, early access, Aug. 16, 2023, doi: [10.1109/JIOT.2023.3305585](https://doi.org/10.1109/JIOT.2023.3305585).

- [26] V. Rimmer, D. Preuveneers, M. Juarez, T. Van Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," in *Proc. 25th Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2018, pp. 1–15.
- [27] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 1928–1943.
- [28] M. Shen, Y. Liu, L. Zhu, X. Du, and J. Hu, "Fine-grained webpage fingerprinting using only packet length information of encrypted traffic," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2046–2059, 2021.
- [29] P. Sirinam, N. Mathews, M. S. Rahman, and M. Wright, "Triplet fingerprinting: More practical and portable website fingerprinting with n-shot learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1131–1148.
- [30] I. Guarino, C. Wang, A. Finamore, A. Pescapè, and D. Rossi, "Many or few samples? comparing transfer, contrastive and meta-learning in encrypted traffic classification," 2023, *arXiv:2305.12432*.
- [31] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, 1st Quart., 2023.
- [32] A. Bagnall, A. Bostrom, J. Large, and J. Lines, "The great time series classification bake off: An experimental evaluation of recently proposed algorithms. extended version," 2016, *arXiv:1602.01711*.
- [33] M. Müller, "Dynamic time warping," in *Information Retrieval for Music and Motion*. Berlin, Germany: Springer, 2007, pp. 69–84.
- [34] Y.-S. Jeong, M. K. Jeong, and O. A. Omitaomu, "Weighted dynamic time warping for time series classification," *Pattern Recognit.*, vol. 44, no. 9, pp. 2231–2240, Sep. 2011. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S003132031000484X>
- [35] P.-F. Marteau, "Time warp edit distance with stiffness adjustment for time series matching," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 31, no. 2, pp. 306–318, Feb. 2009.
- [36] J. Lin, R. Khade, and Y. Li, "Rotation-invariant similarity in time series using bag-of-patterns representation," *J. Intell. Inf. Syst.*, vol. 39, no. 2, pp. 287–315, Oct. 2012. [Online]. Available: <http://link.springer.com/10.1007/s10844-012-0196-5>
- [37] M. G. Baydogan, G. Runger, and E. Tuv, "A bag-of-features framework to classify time series," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 11, pp. 2796–2802, Nov. 2013.
- [38] L. Ye and E. Keogh, "Time series shapelets: A novel technique that allows accurate, interpretable and fast classification," *Data Min. Knowl. Discov.*, vol. 22, pp. 149–182, Jan. 2011.
- [39] T. Rakthanmanon and E. Keogh, "Fast shapelets: A scalable algorithm for discovering time series shapelets," in *Proc. SIAM Int. Conf. Data Min.*, 2013, pp. 668–676.
- [40] A. Bostrom and A. Bagnall, "Binary shapelet transform for multiclass time series classification," in *Proc. 17th Int. Conf., Big Data Anal. Knowl. Discov.*, 2015, pp. 257–269.
- [41] M. Middlehurst, J. Large, M. Flynn, J. Lines, A. Bostrom, and A. Bagnall, "HIVE-COTE 2.0: A new meta ensemble for time series classification," 2021, *arXiv:2104.07551*.
- [42] S. Baik, M. Choi, J. Choi, H. Kim, and K. M. Lee, "Meta-learning with adaptive hyperparameters," 2020, *arXiv:2011.00209*.
- [43] S. Baik, J. Choi, H. Kim, D. Cho, J. Min, and K. M. Lee, "Meta-learning with task-adaptive loss function for few-shot learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 9465–9474. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2021/html/Baik_Meta-Learning_With_Task-Adaptive_Loss_Function_for_Few-Shot_Learning_ICCV_2021_paper.html
- [44] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, 2016, pp. 407–414. [Online]. Available: <http://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0005740704070414>
- [45] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of tor traffic using time based features," in *Proc. ICISp*, 2017, pp. 253–262.
- [46] 2015, "Stratosphere laboratory Datasets," Stratosphere, Accessed: Mar. 13, 2020. [Online]. Available: <https://www.stratosphereips.org/datasets-overview>
- [47] C. Coldwell et al., "Machine learning 5G attack detection in programmable logic," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, 2022, pp. 1365–1370.
- [48] C. Wang et al., "Fingerprinting encrypted voice traffic on smart speakers with deep learning," in *Proc. 13th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, 2020, pp. 254–265. [Online]. Available: <https://dl.acm.org/doi/10.1145/3395351.3399357>
- [49] J. F. I. C. W. Shbair and T. Cholez, 2016, "HTTPS Websites Dataset," betternet. [Online]. Available: <http://betternet.lhs.loria.fr/datasets/https/>
- [50] A. Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," in *Proc. 33rd Adv. Neural Inf. Process. Syst.*, 2019, pp. 1–12.
- [51] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.
- [52] T. Shapira and Y. Shavitt, "The source code of FlowPic," 2021, Accessed: Sep. 2023. [Online]. Available: <https://github.com/talshapira/FlowPic>
- [53] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "The source code of FS-net," 2019, Accessed: Sep. 2023. [Online]. Available: <https://github.com/WSPTTH/FS-Net>
- [54] C. Wang, J. Dani, X. Li, X. Jia, and B. Wang, "The source code of AF," 2021, Accessed: Sep. 2023. [Online]. Available: <https://github.com/SmartHomePrivacyProject/AdaptiveFingerprinting>
- [55] X. Meng, Y. Wang, R. Ma, H. Luo, X. Li, and Y. Zhang, "The source code of PacRep," 2022, Accessed: Sep. 2023. [Online]. Available: <https://github.com/ict-net/PacRep>



Jianjin Zhao (Graduate Student Member, IEEE) received the B.Eng. degree from the Beijing University of Posts and Telecommunications, China, in 2019, where he is currently pursuing the Ph.D. degree in cyberspace security. His current research interests include encrypted traffic analysis and audit log analysis.



Qi Li received the Ph.D. degree in computer science and technology from the Beijing University of Posts and Telecommunications, Beijing, China, in 2010, where she is currently a Professor with the National Engineering Laboratory for Mobile Network Technologies, School of Cyberspace Security. Her current research focuses on information system security.



Yueping Hong received the B.Eng. degree from the Beijing University of Posts and Telecommunications, China, in 2021, where he is currently pursuing the master's degree in cyberspace security. His current research interests include encrypted traffic analysis and graph neural networks.



Meng Shen (Member, IEEE) received the B.Eng. degree in computer science from Shandong University, Jinan, China, in 2009, and the Ph.D. degree in computer science from Tsinghua University, Beijing, China in 2014. He is a Professor with the Beijing Institute of Technology, Beijing. He has authored over 50 papers in top-level journals and conferences, such as ACM SIGCOMM, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, and IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY. His research interests include data privacy and security, blockchain applications, and encrypted traffic classification. He received the Best Paper Runner-Up Award at IEEE IPCCC 2014 and IEEE/ACM IWQoS 2020. He was selected by the Beijing Nova Program 2020 and was the winner of the ACM SIGCOMM China Rising Star Award in 2019. He has guest edited special issues on emerging technologies for data security and privacy in IEEE NETWORK and IEEE INTERNET-OF-THINGS JOURNAL.